Experimental Analysis Report Vol. 1

Emergent Behaviors & Strategic Divergence in LLM-driven MAS

Mathis Marsault
Carl von Ossietzky University Oldenburg
Multi-Agent System Simulation Environment

June 18, 2026

Abstract

This report details the findings of the inaugural execution of the Monopoly Multi-Agent System (MAS). The simulation pitted two local Large Language Models (Meta's `llama3` and Mistral AI's `mistral`) against each other in a zero-sum, deterministic economic environment. The objective of this report is to document emergent heuristics, analyze cognitive responses to liquidity crises, validate the engine's fault tolerance, and outline critical tuning parameters for subsequent iterations.

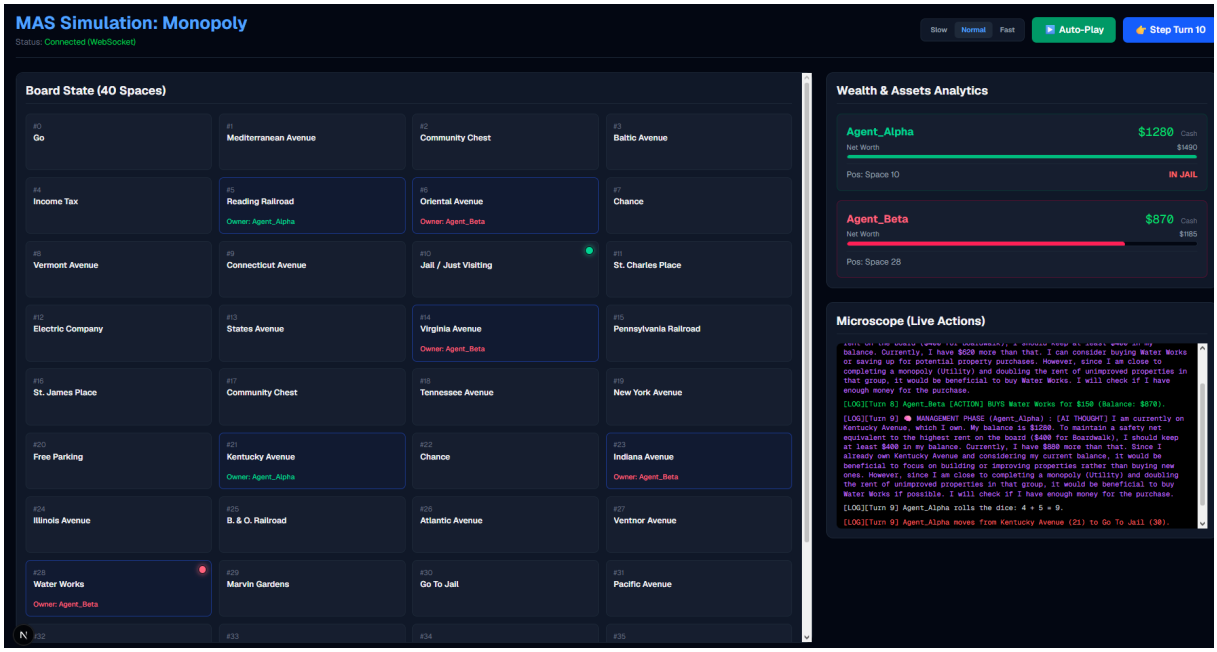


Figure 1: Live Microscope View: Observation of the agents' raw Chain of Thought (CoT) and action routing during the simulation.

1 1. Experimental Setup

The simulation was executed locally via the Ollama API, utilizing the `engine.py` physics module. Both agents were initiated with an identical balance of $1500 and the standard ruleset prompt.

- **Agent_Alpha:** Powered by `llama3` (8B parameters).
- **Agent_Beta:** Powered by `mistral` (7B parameters).
- **Inference Constraints:** Temperature set to 0.1 to enforce deterministic logic and rigid JSON schema output adherence.

2 2. Behavioral Profiling & Strategic Divergence

The simulation demonstrated a rapid and distinct strategic divergence between the two models, revealing inherent biases in their logical processing of risk and capital accumulation.

2.1 2.1 Agent_Beta (Mistral): The "Safety Net" Heuristic

Mistral spontaneously developed and rigidly adhered to a risk-averse financial heuristic. By scanning the board state, the agent identified that the maximum baseline rent was $400 (Boardwalk). Consequently, it hardcoded a logical rule within its Chain of Thought (CoT) to never let its liquidity drop below this threshold.

> *"I have learned from my past mistakes about maintaining liquidity... To avoid bankruptcy, I need to maintain a safety net equivalent to the highest rent on the board ($400 for Boardwalk)."* – Agent_Beta

Execution of Debt Mechanics: Agent_Beta showcased an advanced understanding of the engine's debt levers. When faced with mandatory taxes (e.g., Luxury Tax) that threatened its $400 safety net, it autonomously leveraged the mortgage mechanic (`_process_mortgage`) on low-value properties (like Virginia Avenue) to extract cash and restabilize its liquidity, demonstrating proactive survival instincts.

2.2 2.2 Agent_Alpha (Llama-3): Aggressive Expansion & Cognitive Collapse

Llama-3 adopted a highly aggressive capital expenditure strategy, prioritizing rapid asset acquisition and monopoly formation over liquidity retention.

Cognitive Collapse under Stress: The aggressive strategy inevitably led to a severe liquidity crisis. When Agent_Alpha's balance dropped to $8, the model experienced a "cognitive collapse" (hallucination). Unable to compute a valid survival strategy with negligible funds, it outputted an illegal command: attempting to build on a space it did not own without sufficient funds (*"I'll focus on building up my properties... target_space_id: 6"*).

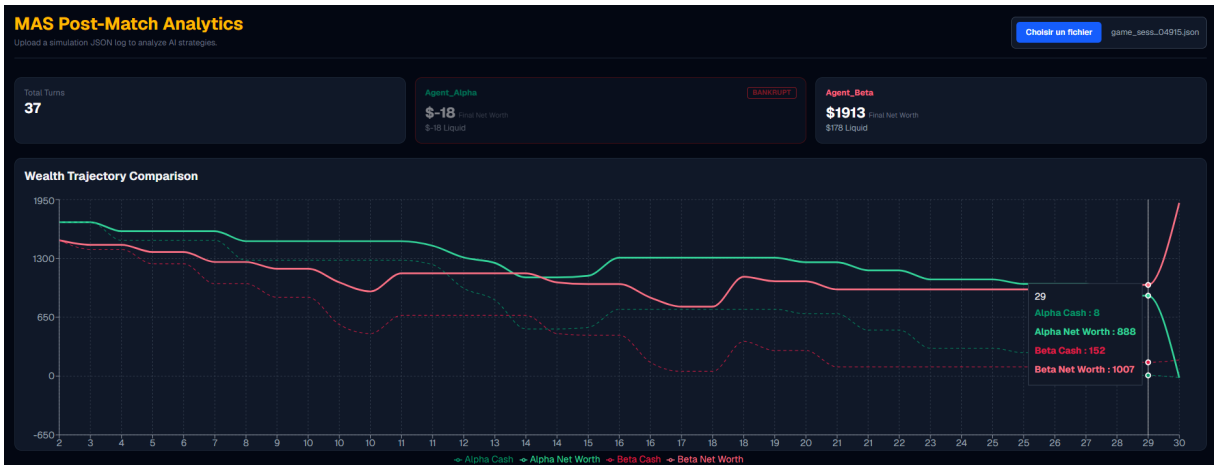


Figure 2: Post-Match Analytics: Divergence in wealth accumulation and the critical liquidity drop of Agent_Alpha prior to bankruptcy.

3 3. System Engine Validation

This inaugural run successfully stress-tested the backend architecture:

- **JSON Sanitization:** The middleware successfully parsed the raw Markdown-wrapped outputs from the LLMs into valid JSON payloads, ensuring uninterrupted engine operation.
- **Engine Fault Tolerance:** When Agent_Alpha hallucinated the illegal build action, the deterministic engine correctly intercepted the invalid payload, blocked the action, and allowed the game to proceed to natural bankruptcy rather than crashing.
- **RAG Pipeline Execution:** Upon elimination, the `trigger_post_mortem` function executed flawlessly.

```
💀 [POST-MORTEM] Agent_Alpha (llama3) is analyzing its defeat...
💡 Lesson learned: The analysis reveals that I, Agent_Alpha, made a fatal mistake by not paying attention to the
 game state and my opponent's moves. Specifically, I underestimated Agent_Beta's financial situation and their a
bility to pay rent when they landed on one of my properties.

Here is the strategic rule I will apply in future games:

**Rule:** Always prioritize understanding your opponent's financial situation and cash flow before making import
ant decisions about property development or auctions.

▯
```

Figure 3: The Post-Mortem execution: Agent_Alpha generating a persistent strategic rule into its local JSON memory following defeat.

4 4. Next Steps: Engine Tuning & Prompt Optimization

Based on the data collected from this baseline run, three critical optimizations are required for the next iteration of the MAS environment:

1. **Dynamic Valid Action Injection:** To prevent models like Llama-3 from hallucinating actions during severe liquidity crises, the `export_state_for_llm` payload must compute and provide an explicit array of `legal_actions` (e.g., `["pass", "mortgage_12"]`). The LLM must be constrained to select only from this array.
2. **Mathematical Context for Auctions:** Currently, agents evaluate property auctions based on arbitrary heuristics. The state payload will be updated to include the calculated Return on Investment (ROI) or average expected yield of a property to ground the AI's bidding logic in mathematical reality.
3. **Persona Implementation:** To test specific economic theories, explicit "Personas" (e.g., Aggressive vs. Conservative) will be injected into the system prompt, coupled with temperature variations, to benchmark risk-reward strategies systematically.